

Deployment, Operations & Recovery Runbook

UCO Seed Scripts · Kubernetes/Helm · Ed25519 Key Ceremony · Flyway V1-V4 · WAL Replication · WORM Verification · Post-Deployment Checklist

Version:	1.0 May 2026
Date Generated:	Friday, 22 May 2026 — 15:39 CDT
Prepared by:	SMEPro Technologies Engineering
Contact:	cmiguez@smeprotech.com
Series:	IOS+ Engineering Body, Document 6 of 6
Author (Operator):	Chris — SMEPro Technologies Engineering Houston, TX

Section 0 — Engineering Body Series Index

The IOS+ Engineering Body is a six-document specification set that defines the complete design, data model, compliance logic, security architecture, and operational runbook for the IOS+ platform. All six documents are classified CONFIDENTIAL and require Enhanced NDA clearance. Documents are intended to be

read in order 1-5 for design comprehension; Document 6 is executed in operational sequence.

Doc #	Artifact ID	Title	Scope
EB Doc 1	IOS-EB-001	System Architecture & Module Design	YBR-L4 four-tier architecture, IOS+ Middleware Engine layers, Gate 530 sidecar pattern, service mesh topology, and pod/node layout specifications.
EB Doc 2	IOS-EB-002	Evidence Fabric & Cryptographic Signing Architecture	Ed25519 signing model, triple-publication protocol, 90-day key rotation schedule, Evidence Package structure, and WORM enforcement design.
EB Doc 3	IOS-EB-003	COS+ Database Schema & Access Control	Flyway migration definitions V1-V4, PostgreSQL role model, WORM trigger SQL, audit table schemas, UCO schema extension tables, and row-level security policy.
EB Doc 4	IOS-EB-004	Gate 530 UCO Compliance Dimension Matrix	Universal Compliance Decoding Matrix across 19 industry sectors plus XSC cross-cutting partition; 350-node dataset with risk weights, policy actions, and enforcement logic.
EB Doc 5	IOS-EB-005	RAG Vault Vector Search & Retrieval	20-partition HNSW index architecture,

Doc #	Artifact ID	Title	Scope
		Design	sector-specific ef_search and ef_construction parameters, embedding pipeline design, overdue injection logic, and retrieval scoring.
EB Doc 6	IOS-EB-006	Deployment, Operations & Recovery Runbook (this document)	Step-by-step execution of all deployment procedures: Flyway migrations, role setup, Ed25519 key ceremony, UCO seed data loading, Kubernetes/Helm deployment, WAL replication, WORM verification, post-deployment validation, key rotation, observability, and recovery.

Series Navigation Note

EB Doc 6 (this document) is the operational runbook. It is the last document to be read and the first to be executed.

Section 1 — Deployment Prerequisites

1.1 Infrastructure Requirements

The following minimum production cluster specification maps directly to the YBR-L4 four-tier architecture defined in EB Doc 1. All nodes must be provisioned and labelled before proceeding with any deployment step.

Tier 2 — Orchestration (IOS+ Middleware Engine + Gate 530 Sidecar)

Parameter	Specification
Node count	3× nodes
CPU	16 vCPU per node
RAM	64 GB per node
Storage	500 GB NVMe OS disk per node
Kubernetes label	tier=orchestration

Tier 3 — Inference (RAG Vault Vector Search)

Parameter	Specification
Node count	2× nodes
RAM	32 GB per node
CPU	8 vCPU per node
Storage	1 TB NVMe for HNSW index hot storage
GPU (optional)	A10G or T4 for embedding generation acceleration
Kubernetes label	tier=inference

Tier 4 — Persistence (COS+ Database Cluster)

Parameter	Primary	Hot Standby
CPU	32 vCPU	32 vCPU (identical to primary)
RAM	128 GB	128 GB (identical to primary)
Storage	6× 4 TB NVMe (RAID-10)	6× 4 TB NVMe (RAID-10)
Kubernetes label	tier=persistence	tier=persistence

Additional Infrastructure

Component	Specification
HashiCorp Vault cluster	3-node HA, 16 GB RAM each
Redis cluster	3-node HA for Gate 530 session cache, 32 GB RAM each
Load balancer	Layer 7, TLS termination required
DNS zone management	API access required for Ed25519 key publication (TXT records)

1.2 Software Prerequisites

Software	Required Version	Notes
Kubernetes	1.29+	Cluster must be accessible via <code>kubectl</code>
Helm	3.14+	Must be authenticated to chart registry
Flyway	10.x	CLI installed on deployment host
PostgreSQL	16.x	Extensions: <code>pgcrypto</code> , <code>pgvector</code> , <code>btree_gist</code>
HashiCorp Vault	1.16+	Transit engine enabled; <code>ios-plus/</code> secrets path mounted
Redis	7.2+	Cluster mode enabled
CLIs	Current stable	<code>kubectl</code> , <code>psql</code> , <code>vault</code>

Software	Required Version	Notes
		configured with cluster access
AWS CLI / S3-compatible	Current stable	Required for WAL archiving to object store

1.3 Secrets Management

All passwords, connection strings, API keys, and private key material are injected via HashiCorp Vault Agent sidecar into Kubernetes Secrets at pod startup. No secrets are stored in Helm values files, ConfigMaps, or container images. Verify Vault is unsealed and the `ios-plus/` secrets engine path is mounted before proceeding with any step in this runbook.

Warning — Secrets Hygiene

Any plaintext password found in a Helm values file, ConfigMap, shell history, or container image must be treated as compromised and rotated immediately. All credential access must be traceable through the Vault audit log.

Section 2 — Flyway Migration Execution Sequence (V1-V4)

2.1 Migration Overview

Version	Filename	Description	Est. Runtime	Rollback Risk
V1	<code>v1__initial_schema.sql</code>	Core operational tables: objects,	~5s	Low

Version	Filename	Description	Est. Runtime	Rollback Risk
		tenant_registry, regulatory_profiles, ios_signing_keys		
V2	V2__audit_tables.sql	Audit tables with WORM triggers: evidence_packages, gate_decisions, evidence_source_manifest, quarantine_records, merkle_roots	~8s	Low (no data yet)
V3	V3__rag_vault_schema.sql	RAG Vault tables: rag_sources, rag_chunks (partitioned), HNSW indexes	~120s (HNSW index build)	Low
V4	V4__uco_schema_extension.sql	UCO Amendment v1.1: agency_registry, uco_nodes, naics_decoder, code_crosswalk, compliance_chains, tenant_naics_profiles, uco_evaluation_results, filing_calendar, rag_vault_sector_partitions	~30s + seed data insert	Medium (seed data)

2.2 Pre-Migration Checklist

Execute all four verification steps below before running ANY migration. Do not proceed if any step fails.

```
# 1. Verify target database is the correct environment psql $COS_DB_URL -c
"SELECT current_database(), current_user, version();" # 2. Verify extensions
are installed psql $COS_DB_URL -c "SELECT extname, extversion FROM
pg_extension WHERE extname IN ('pgcrypto','vector','btree_gist');" #
Expected: 3 rows returned # 3. Take a pre-migration backup pg_dump --
format=custom --no-acl --no-owner $COS_DB_URL > backups/pre_migration_$(date
+%Y%m%d_%H%M%S).dump echo "Pre-migration backup complete: $?" # 4. Confirm
Flyway baseline matches current schema version flyway -url=$COS_DB_URL info
```

2.3 Execute Migrations V1 → V4 in Sequence

```
# Set environment export
COS_DB_URL="jdbc:postgresql://${DB_HOST}:5432/cos_plus?sslmode=require"
export FLYWAY_USER="cos_admin" # Run all migrations flyway \ -
url="$COS_DB_URL" \ -user="$FLYWAY_USER" \ -password="$(vault kv get -
field=password secret/ios-plus/db/cos_admin)" \
-locations="filesystem:db/migrations" \ -validateOnMigrate=true \ migrate
# Verify all migrations applied flyway \ -url="$COS_DB_URL" \ -
user="$FLYWAY_USER" \ -password="$(vault kv get -field=password secret/ios-
plus/db/cos_admin)" \ info # Expected output: V1, V2, V3, V4 all showing
Success status
```

2.4 Post-V2 WORM Trigger Verification

Immediately after the V2 migration completes, run the WORM verification test (defined in EB Doc 3 §7.3) to confirm append-only enforcement is active before any data is written to audit tables. Do not proceed to V3 until this step passes.

```
psql $COS_DB_URL -U cos_admin -f scripts/db/verify_worm_triggers.sql #
Expected output: WORM TEST PASSED for all audit tables # If any FAILED
message appears: DO NOT PROCEED – investigate trigger installation
```

Stop Condition — Post-V2 WORM Check

If any WORM trigger test returns a FAILED status, halt the deployment immediately. Do not proceed to V3 or V4. Escalate to Engineering Lead. A failed WORM trigger means audit tables accept UPDATE and DELETE — the entire evidence integrity model is broken until resolved.

Section 3 — PostgreSQL Role Setup

3.1 Role Creation Script

Execute after V1 migration completes. Passwords are fetched from Vault at runtime — never hardcoded in SQL files or shell scripts.

```
-- Run as superuser (cos_admin) on fresh COS+ database -- Application
read/write role for IOS+ Middleware Engine CREATE ROLE ios_app WITH LOGIN;
COMMENT ON ROLE ios_app IS 'Primary application role. SELECT/INSERT on
operational tables. No audit log access.'; -- Audit INSERT-only role for
Evidence Fabric signing service CREATE ROLE audit_writer WITH LOGIN; COMMENT
ON ROLE audit_writer IS 'INSERT-only on all audit tables. UPDATE/DELETE
explicitly revoked. Evidence Fabric exclusive.'; -- Audit SELECT-only role
for compliance review CREATE ROLE audit_reader WITH LOGIN; COMMENT ON ROLE
audit_reader IS 'SELECT-only on audit tables. Compliance review API and
external auditors.'; -- Vector corpus read role for RAG Vault retrieval
CREATE ROLE rag_reader WITH LOGIN; COMMENT ON ROLE rag_reader IS 'SELECT on
RAG Vault tables and uco_nodes vector index.'; -- RAG Vault ingestion write
role CREATE ROLE rag_writer WITH LOGIN; COMMENT ON ROLE rag_writer IS
'INSERT/UPDATE on rag_sources and rag_chunks. Ingestion pipeline exclusive.';
```

3.2 Password Injection from Vault

```
# Inject passwords – NEVER set passwords in plain SQL files for ROLE in
ios_app audit_writer audit_reader rag_reader rag_writer; do PASSWORD=$(
vault kv get -field=password secret/ios-plus/db/$ROLE) psql $COS_DB_URL -U
cos_admin \      -c "ALTER ROLE $ROLE WITH PASSWORD '$PASSWORD';" echo
"Password set for $ROLE" done
```

3.3 Grant Privileges (Post V4 Migration — All Tables Exist)

```
psql $COS_DB_URL -U cos_admin -f db/grants/V4_role_grants.sql # This script
applies all GRANT and REVOKE statements from EB Doc 3 # Sections 2.2, 2.3,
and 6.13 – roles for base tables AND UCO extension tables
```

3.4 Role Verification

```
-- Verify ios_app cannot touch audit tables SET ROLE ios_app; SELECT COUNT(*)
FROM evidence_packages; -- Should raise: permission denied -- Verify
audit_writer cannot UPDATE audit records SET ROLE audit_writer; UPDATE
evidence_packages SET output_hash = 'x' WHERE evidence_package_id =
gen_random_uuid(); -- Should raise: restrict_violation (WORM trigger fires
before privilege check) -- Verify rag_reader can query UCO vector index SET
ROLE rag_reader; SELECT uco_node_id, governing_agency FROM uco_nodes LIMIT 5;
-- Should return rows RESET ROLE;
```

Section 4 — Ed25519 Key Generation Ceremony

This section defines the air-gapped key generation ceremony for IOS+ signing keys. This procedure is performed **once** at initial deployment and thereafter only during planned key rotation (90-day cycle per EB Doc 2 §2.3).

Four-Eyes Principle — Mandatory

No single operator may complete the key ceremony alone. Two authorized operators must be physically present and recorded in the physical access log for the entire duration of steps 4.2 through 4.5. Ceremony records are subject to audit review.

4.1 Ceremony Prerequisites

- Air-gapped laptop (no network interfaces active — physically verified before starting)
- Two authorized operators (four-eyes principle)
- USB drive with `libsodium` CLI and ceremony scripts (pre-vetted, SHA-256 checksummed)
- Physical access log: record operator names, date, time, location before any action
- Hardware security token for operator authentication (YubiKey 5 or equivalent)

4.2 Key Generation

```
# On air-gapped system — run as ceremony operator # Generate Ed25519 key pair
using libsodium python3 ceremony/generate_ed25519_keypair.py \  --purpose
evidence_signing \  --tenant global \  --output-dir /media/usb/keys/$(date
+%Y%m%d)/ # Script output: # Private key seed: [32 bytes] → printed to paper
+ sealed in tamper-evident envelope # Public key: [32 bytes base64url] →
written to public_key.txt # Key fingerprint (SHA-256 of public key bytes): →
written to fingerprint.txt # Key ID (UUID v4): → written to key_id.txt #
Verify key pair is valid python3 ceremony/verify_keypair.py --dir
```

```
/media/usb/keys/$(date +%Y%m%d)/ # Expected: "KEYPAIR VALID – Ed25519  
sign/verify round-trip passed"
```

4.3 Private Key Import to HashiCorp Vault

```
# Transfer public_key.txt and key_id.txt to network-connected system (USB) #  
Private key seed is NEVER transferred electronically – remains on air-gapped  
system # On deployment system – import via Vault transit engine vault write  
transit/keys/ios-evidence-signing \ type="ed25519" \ exportable=false \  
allow_plaintext_backup=false # Store key metadata in Vault KV vault kv put  
secret/ios-plus/signing/evidence-signing \ key_id="$(cat key_id.txt)" \  
public_key_b64url="$(cat public_key.txt)" \ public_key_fingerprint="$(cat  
fingerprint.txt)" \ purpose="evidence_signing" \ activates_at="$(date -u  
+%Y-%m-%dT%H:%M:%SZ --date='+1 hour')" \ expires_at="$(date -u +%Y-%m-%dT  
%H:%M:%SZ --date='+91 days')" \ created_by="$(whoami)" echo "Key metadata  
stored in Vault"
```

4.4 Triple-Publication of Verification Key

All three publication locations must be updated atomically. A partial publication (fewer than 3 locations) constitutes an incomplete ceremony and must be remediated before starting the IOS+ Middleware Engine.

Publication 1 — COS+ Database

```
INSERT INTO ios_signing_keys ( key_id, public_key_bytes, public_key_b64url,  
public_key_fingerprint, purpose, tenant_id, activates_at, expires_at,  
created_by_operator ) VALUES ( $KEY_ID, decode($PUBLIC_KEY_B64URL,  
'base64'), $PUBLIC_KEY_B64URL, $FINGERPRINT, 'evidence_signing',  
NULL, -- global key NOW() + INTERVAL '1 hour', NOW() + INTERVAL '91  
days', $OPERATOR_NAME );
```

Publication 2 — DNS TXT Record

```
# Publish to DNS zone via API (Route53 example) aws route53 change-resource-  
record-sets \ --hosted-zone-id $ZONE_ID \ --change-batch  
{  
  "Changes": [{  
    "Action": "UPSERT",  
    "ResourceRecordSet": {  
      "Name": "_ios-signing-key.smeprotech.com",  
      "Type": "TXT",  
      "TTL": 300,  
      "ResourceRecords": [{  
        "Value": "\"_ios-signing-  
key-v1 kid='\"$KEY_ID\"' pk='\"$PUBLIC_KEY_B64URL\"'\"  
fp='\"$FINGERPRINT\"' exp='\"$EXPIRES_UNIX\"'\" } ] }  
}] }' # Verify DNS propagation dig TXT _ios-signing-key.smeprotech.com  
+short
```

Publication 3 — Deployment Filesystem

```
# Write public key to well-known path echo "$PUBLIC_KEY_B64URL" > /etc/ios-  
plus/signing-keys/${KEY_ID}.pub echo  
"{\"key_id\": \"$KEY_ID\", \"fingerprint\": \"$FINGERPRINT\", \"activates_at\": \"$  
ACTIVATES_AT\", \"expires_at\": \"$EXPIRES_AT\"}" \ > /etc/ios-plus/signing-  
keys/${KEY_ID}.meta.json # Update current symlink atomically ln -sf  
/etc/ios-plus/signing-keys/${KEY_ID}.pub \ /etc/ios-plus/signing-  
keys/current.pub # Verify all three publication locations match python3  
scripts/ops/verify_key_publication_consistency.py \ --expected-key-id
```

```
$KEY_ID \ --expected-fingerprint $FINGERPRINT # Expected: "ALL THREE KEY PUBLICATION LOCATIONS CONSISTENT ✓"
```

4.5 Ceremony Completion Record

```
# Write signed ceremony record to audit log psql $COS_DB_URL -U audit_writer -c " INSERT INTO merkle_roots (window_start, window_end, record_count, merkle_root_hash, ed25519_signature, signing_key_id, published_to_dns) VALUES (NOW(), NOW(), 0, 'KEY_CEREMONY_$(date +%Y%m%d)', 'CEREMONY_PLACEHOLDER', '$KEY_ID', TRUE); " echo "Key ceremony complete. Physical records sealed and logged."
```

Section 5 — UCO Seed Data Deployment

5.1 Seed Execution Order

UCO seed data must be loaded in strict foreign key dependency order. Do **not** parallelise these steps. Each step must complete and be verified before proceeding to the next.

Step	Table	Dependency Note
1	agency_registry	No FK dependencies on new tables
2	naics_decoder	Standalone; references agency_registry conceptually via TEXT[]
3	uco_nodes	Primary UCO table — 350 rows. Critical path.
4	code_crosswalk	Standalone; references uco_node_ids as TEXT[]
5	compliance_chains	FK to uco_nodes + agency_registry
6	rag_vault_sector_partitions	Standalone; already seeded by V4 migration INSERT block

5.2 Agency Registry Seed (Step 1)

```
# Load from canonical seed CSV psql $COS_DB_URL -U cos_admin \ -c "\COPY
agency_registry FROM 'db/seeds/agency_registry.csv' CSV HEADER" # Verify
count psql $COS_DB_URL -c "SELECT COUNT(*) FROM agency_registry;" # Expected:
>= 80 rows
```

Spot-check key agencies after load:

```
SELECT agency_code, full_name, agency_type, jurisdiction FROM agency_registry
WHERE agency_code IN
('EPA','OSHA','CMS','FinCEN','FERC','NRC','FDIC','CFPB','FCC','DEA','FDA')
ORDER BY agency_code; -- Expected: 11 rows, all present
```

5.3 NAICS Decoder Seed (Step 2)

```
psql $COS_DB_URL -U cos_admin \ -c "\COPY naics_decoder FROM
'db/seeds/naics_decoder.csv' CSV HEADER" psql $COS_DB_URL -c "SELECT
COUNT(*) FROM naics_decoder;" # Expected: 100+ rows covering all NAICS 3-
digit subsectors relevant to UCO sectors
```

5.4 UCO Nodes Seed — 350 Rows (Step 3 — CRITICAL)

```
# Load the 350-node UCO dataset from the canonical seed file # Source: SMEPro
Universal Compliance Decoding Matrix (all 19 sector sheets + XSC) psql
$COS_DB_URL -U cos_admin \ -c "\COPY uco_nodes (uco_node_id, sector_code,
broad_industry, industry_subtype, specific_activity, jurisdiction_level,
governing_agency, regulation_name, cfr_usc_citation, report_form_name,
form_code, filing_frequency, key_due_dates, business_segment,
penalties_consequences, cip_code, sic_code, naics_code, soc_code, isic_code,
hs_hts_code, notes, ontology_level, compliance_chain_ref, operating_segment,
responsible_role, enforcement_type, risk_weight, ybr_gate, policy_action,
is_cross_cutting, last_updated) FROM 'db/seeds/uco_nodes_350.csv' CSV HEADER"
# MANDATORY VERIFICATION — DO NOT SKIP psql $COS_DB_URL -c "SELECT COUNT(*)
FROM uco_nodes;" # Expected: 350 psql $COS_DB_URL -c " SELECT sector_code,
COUNT(*) AS node_count, MIN(risk_weight) AS min_risk, MAX(risk_weight) AS
max_risk, COUNT(*) FILTER (WHERE policy_action='BLOCK') AS block_count,
COUNT(*) FILTER (WHERE policy_action='APPROVE') AS approve_count, COUNT(*)
FILTER (WHERE policy_action='ESCALATE') AS escalate_count FROM uco_nodes
GROUP BY sector_code ORDER BY sector_code; " # Expected totals: BLOCK=192,
APPROVE=108, ESCALATE=50 # Expected: sector XSC has 19 rows with
is_cross_cutting=TRUE # Verify no node has risk_weight < 5 (matrix floor)
psql $COS_DB_URL -c "SELECT COUNT(*) FROM uco_nodes WHERE risk_weight < 5;" #
Expected: 0
```

CRITICAL — Gate 530 Compliance Failure Risk

An empty or partially-seeded `uco_nodes` table will cause Gate 530 to evaluate zero compliance dimensions, producing incorrectly permissive APPROVE decisions for ALL queries. This is a catastrophic compliance failure. Verify the count is **exactly 350** before starting the IOS+ Middleware Engine. Do not

proceed if the count differs from 350 for any reason.

5.5 Code Crosswalk Seed (Step 4)

```
psql $COS_DB_URL -U cos_admin \ -c "\COPY code_crosswalk FROM
'db/seeds/code_crosswalk.csv' CSV HEADER" psql $COS_DB_URL -c "SELECT
COUNT(*) FROM code_crosswalk;" # Expected: 200+ rows # Verify GIN index is
active and populated psql $COS_DB_URL -c " SELECT indexname, indexdef FROM
pg_indexes WHERE tablename = 'code_crosswalk' AND indexname =
'idx_code_crosswalk_uco_node_ids'; "
```

5.6 Compliance Chains Seed (Step 5)

```
psql $COS_DB_URL -U cos_admin \ -c "\COPY compliance_chains (uco_node_id,
cip_code, naics_code, jurisdiction_level, agency_code, regulation_name,
chain_text, chain_hash) FROM 'db/seeds/compliance_chains.csv' CSV HEADER" #
Verify no duplicate chain hashes (UNIQUE constraint should prevent, but
verify) psql $COS_DB_URL -c " SELECT chain_hash, COUNT(*) AS cnt FROM
compliance_chains GROUP BY chain_hash HAVING COUNT(*) > 1; " # Expected: 0
rows (no duplicates)
```

5.7 UCO Node Vector Embeddings (Step 6 — Async, Post-Startup)

UCO node vector embeddings are generated by the embedding service on first startup, not seeded from CSV. Embeddings are large and environment-specific. The embedding service runs as a Kubernetes Job after initial deployment.

```
kubectl apply -f infra/kubernetes/jobs/uco-embedding-job.yaml # Job: for each
uco_node, concatenate regulation_name + specific_activity + cfr_usc_citation
+ penalties_consequences # → generate text-embedding-3-small embedding (1536
dims) via OpenAI API # → UPDATE uco_nodes SET vector_embedding = $embedding
WHERE uco_node_id = $id # Note: requires ios_app role UPDATE on
uco_nodes.vector_embedding column (one-time exception, revoked after job)
kubectl wait --for=condition=complete job/uco-embedding-job --timeout=1800s #
Expected runtime: ~15 minutes for 350 nodes at batch size 100 # Verify
embeddings populated psql $COS_DB_URL -c "SELECT COUNT(*) FROM uco_nodes
WHERE vector_embedding IS NOT NULL;" # Expected: 350
```

Section 6 — Kubernetes and Helm Deployment

6.1 Helm Chart Structure

```
infra/helm/ios-plus/ |— Chart.yaml |— values.yaml # Non-secret
configuration – safe for version control |— values.production.yaml #
Production overrides – non-secret |— templates/ | |— middleware-engine/
# Deployment, Service, HPA | |— gate-530/ # DaemonSet sidecar
template | |— evidence-fabric/ # Deployment, Service | |— rag-
vault/ # Deployment, Service, PVC | |— rag-ingestion/ #
CronJob for scheduled re-ingestion | |— cos-plus/ # StatefulSet
for persistence tier | |— redis/ # StatefulSet for Gate 530
session cache | |— shared/ # ConfigMaps, ServiceAccounts,
RBAC |— ci/ |— deploy.sh # CI/CD deployment script
```

6.2 Core values.yaml

```
global: environment: production region: us-central1 imagePullPolicy:
IfNotPresent imageRegistry: ghcr.io/smeptotech middlewareEngine:
replicaCount: 3 nodeSelector: tier: orchestration resources:
requests: cpu: "4" memory: "16Gi" limits: cpu: "8"
memory: "32Gi" layerTimeouts: l1Ms: 10 l2Ms: 50 l3Ms: 80
l4Ms: 5 l5Ms: 60 # Gate 530 budget l6Ms: 150 # RAG Vault budget
l7Ms: 200 # Synthesis budget gate530: deploymentMode: sidecar # Runs
as sidecar in middleware-engine pod ipcSocketPath: /tmp/gate530.sock
sessionCacheRedisCluster: redis-cluster:6379 sessionCacheTTLSeconds: 14400
failClosedOnTimeout: true # Critical: timeout = BLOCK evidenceFabric:
replicaCount: 2 nodeSelector: tier: orchestration
signingKeyVaultPath: transit/keys/ios-evidence-signing wormCommitTimeoutMs:
25 ragVault: replicaCount: 2 nodeSelector: tier: inference
hnsWEfSearch: critical: 128 # Energy, Healthcare, Finance, Insurance,
Education, Public Admin high: 64 # All HIGH risk-tier sectors
medium: 40 # Agriculture, Accommodation xsc: 64 # Cross-
cutting partition embeddingModel: text-embedding-3-small
embeddingDimensions: 1536 retrievalThreshold: 0.75
overdueInjectionEnabled: true # Always inject OVERDUE UCO nodes cosPlus:
replicaCount: 1 # Primary only; standby managed separately nodeSelector:
tier: persistence walLevel: replica maxWalSenders: 10 walKeepSize: 2GB
checkpointCompletionTarget: 0.9 sharedBuffers: 16GB effectiveCacheSize:
48GB workMem: 64MB vault: address: https://vault.internal:8200 role:
ios-plus-k8s secretPaths: dbIosApp: secret/ios-plus/db/ios_app
dbAuditWriter: secret/ios-plus/db/audit_writer dbAuditReader: secret/ios-
plus/db/audit_reader dbRagReader: secret/ios-plus/db/rag_reader
dbRagWriter: secret/ios-plus/db/rag_writer signingKeyMeta: secret/ios-
plus/signing/evidence-signing
```

6.3 Deploy Command

```
# Deploy to production cluster helm upgrade --install ios-plus
infra/helm/ios-plus \ --namespace ios-plus \ --create-namespace \ --
values infra/helm/ios-plus/values.yaml \ --values
infra/helm/ios-plus/values.production.yaml \ --set global.imageTag=$(git
```

```
rev-parse --short HEAD) \ --atomic \ --timeout 10m \ --wait # Verify
all pods are running kubectl get pods -n ios-plus # Expected: all pods in
Running state # middleware-engine-* (3 pods), evidence-fabric-* (2 pods),
rag-vault-* (2 pods), etc.
```

6.4 Gate 530 Sidecar Verification

```
# Verify Gate 530 IPC socket is active on each middleware-engine pod kubectl
exec -n ios-plus deployment/middleware-engine -- \ ls -la /tmp/gate530.sock
# Expected: srwxrwxrwx ... /tmp/gate530.sock # Confirm Gate 530 session
cache is connected kubectl exec -n ios-plus deployment/gate-530-sidecar -- \
redis-cli -h redis-cluster PING # Expected: PONG
```

Section 7 — WAL Streaming Replication (15-Minute RPO)

7.1 Primary Configuration (postgresql.conf)

```
# WAL settings for 15-minute RPO wal_level = replica max_wal_senders = 10
wal_keep_size = 2GB archive_mode = on archive_command = 'aws s3 cp %p
s3://smeprotech-wal-archive/ios-plus/%f' archive_timeout = 900 #
Force WAL segment archive every 900s = 15 minutes hot_standby = on
synchronous_commit = remote_write # Audit tables: WAL acknowledged by
standby before COMMIT returns
```

7.2 pg_hba.conf for Replication

```
host replication replication_user standby_ip/32 scram-sha-256
```

7.3 Standby Initialization

```
# On standby node – initial basebackup from primary pg_basebackup \ --
host=$PRIMARY_HOST \ --port=5432 \ --username=replication_user \ --
pgdata=/var/lib/postgresql/data \ --format=plain \ --write-recovery-
conf \ --progress \ --checkpoint=fast # Configure recovery cat >
/var/lib/postgresql/data/postgresql.auto.conf << EOF primary_conninfo =
'host=$PRIMARY_HOST port=5432 user=replication_user sslmode=require'
recovery_target_timeline = 'latest' EOF # Start standby pg_ctl start -D
/var/lib/postgresql/data
```

7.4 Replication Lag Monitoring

```
-- Run on primary to verify standby is streaming within RPO SELECT
client_addr, state, sent_lsn, write_lsn, flush_lsn, replay_lsn,
(sent_lsn - replay_lsn) AS replication_lag_bytes, write_lag, flush_lag,
replay_lag FROM pg_stat_replication; -- Alert if replay_lag > INTERVAL '15
minutes'
```


RPO Design Note

The `archive_timeout = 900` setting forces a WAL segment to be archived every 15 minutes even if no new writes have occurred, ensuring the S3 WAL archive is never more than 15 minutes stale. In high-write production environments, WAL segments will archive more frequently.

Section 8 — Recovery Procedures

8.1 Planned Failover (Primary Maintenance)

```
# 1. Verify standby is caught up psql $PRIMARY_URL -c "SELECT replay_lag FROM
pg_stat_replication;" # Must be < 30 seconds before proceeding # 2. Demote
primary (Patroni managed) patronictl -c /etc/patroni/config.yml switchover
ios-plus-cluster \ --master primary-node \ --candidate standby-node \
--force # 3. Verify new primary is accepting writes psql $STANDBY_URL -c
"SELECT pg_is_in_recovery();" # Expected: f (false = no longer in recovery =
now primary) # 4. Update connection strings in Vault vault kv patch
secret/ios-plus/db/ios_app host=$NEW_PRIMARY_HOST # (repeat for all roles) #
5. Rolling restart middleware-engine pods to pick up new connection strings
kubectl rollout restart deployment/middleware-engine -n ios-plus
```

8.2 Unplanned Failover (Primary Failure — <90 Second RTO)

```
# Patroni automatic failover triggers within 30 seconds of primary failure
detection # Monitor via: patronictl -c /etc/patroni/config.yml list ios-plus-
cluster # Verify promotion completed psql $STANDBY_URL -c "SELECT
pg_is_in_recovery();" # Expected: f # Verify WAL archive gap (determines
actual data exposure window) aws s3 ls s3://smeprotech-wal-archive/ios-plus/
| tail -5 # Last archived segment timestamp = actual RPO boundary # Maximum
exposure: 15 minutes (archive_timeout = 900s) # After failover: run evidence
integrity check python3 scripts/ops/verify_evidence_chain_integrity.py \ --
since "$(date -u -d '1 hour ago' +%Y-%m-%dT%H:%M:%SZ)" \ --db-url
$NEW_PRIMARY_URL # This script: fetches all Evidence Packages from last hour,
verifies Ed25519 signatures # Reports any gap in trace_id sequence that may
indicate lost records
```

8.3 Point-in-Time Recovery from WAL Archive

```
# Restore to specific timestamp (e.g., before accidental DDL)
TARGET_TIME="2026-05-22 14:30:00 UTC" pg_restore_from_archive.sh \ --s3-
```

```
prefix s3://smeprotech-wal-archive/ios-plus/ \ --target-time "$TARGET_TIME"
\ --restore-path /var/lib/postgresql/data_pitr # Add recovery target to
postgresql.auto.conf echo "recovery_target_time = '$TARGET_TIME'" >>
/var/lib/postgresql/data_pitr/postgresql.auto.conf echo
"recovery_target_action = 'promote'" >>
/var/lib/postgresql/data_pitr/postgresql.auto.conf
```

Section 9 — WORM Verification Procedures

9.1 Post-Deployment Mandatory WORM Test

Run immediately after V2 migration and again after V4 migration completes. This test must pass before any application traffic is directed to the system.

```
psql $COS_DB_URL -U cos_admin -f scripts/db/verify_worm_triggers.sql
```

The script tests UPDATE and DELETE on all six WORM-protected tables:

Table	Expected Result
evidence_packages	Must raise restrict_violation
gate_decisions	Must raise restrict_violation
evidence_source_manifest	Must raise restrict_violation
quarantine_records	Must raise restrict_violation (except via stored procedure)
uco_evaluation_results	Must raise restrict_violation
merkle_roots	Must raise restrict_violation

Expected final output: WORM VERIFICATION COMPLETE: All append-only constraints confirmed active

STOP — WORM Failure Is a Hard Blocker

Any FAILED message from the WORM verification script requires immediate halt of all deployment activity. Escalate to Engineering Lead immediately. Do not

attempt workarounds. The evidence integrity model is contingent on all six WORM constraints being active simultaneously.

9.2 Weekly Automated WORM Integrity Check

Deployed as a Kubernetes CronJob running every Sunday at 02:00 UTC.

```
apiVersion: batch/v1 kind: CronJob metadata:  name: worm-integrity-check
namespace: ios-plus spec:  schedule: "0 2 * * 0"  jobTemplate:  spec:
template:  spec:  containers:  - name: worm-checker
image: ghcr.io/smeprotech/ios-plus-ops:latest  command:
["python3", "scripts/ops/weekly_worm_check.py"]  env:
- name: DB_URL  valueFrom:  secretKeyRef:
name: cos-plus-audit-reader  key: connection_string
```

9.3 Merkle Root Verification

```
python3 scripts/ops/verify_merkle_root.py \  --window-start "2026-05-
22T00:00:00Z" \  --window-end "2026-05-22T23:59:59Z" \  --db-url
$COS_DB_URL # Computes expected Merkle root from evidence_packages in window
# Compares against stored merkle_roots.merkle_root_hash # Verifies Ed25519
signature on stored root using DNS-published verification key # Expected:
"MERKLE ROOT VERIFIED ✓ – No records inserted or deleted since root was
signed"
```

Section 10 — RAG Vault Partition Initialization

10.1 Create Sector-Partitioned rag_chunks Tables

The V3 migration creates the base `rag_chunks` table with declarative partitioning. Post-migration, create the 20 sector sub-partitions using the partition creation script.

```
psql $COS_DB_URL -U cos_admin -f
db/migrations/V3_partitions/create_sector_partitions.sql
```

The partition creation script creates 20 child tables: `rag_chunks_p01` (Energy) through `rag_chunks_p19` (Management of Companies) plus `rag_chunks_xsc` (Cross-Cutting). Each partition is created with sector-specific HNSW parameters (`m` and `ef_construction`) per EB Doc 5 §2.2.

10.2 Verify All 20 Partitions Exist

```
SELECT child.relname AS partition_table, parent.relname AS parent_table,
pg_size_pretty(pg_total_relation_size(child.oid)) AS size FROM pg_inherits
JOIN pg_class parent ON pg_inherits.inhparent = parent.oid JOIN pg_class
child ON pg_inherits.inhrelid = child.oid WHERE parent.relname = 'rag_chunks'
ORDER BY child.relname; -- Expected: 20 rows (rag_chunks_p01 through
rag_chunks_p19 + rag_chunks_xsc)
```

10.3 Initial Corpus Loading

Load regulatory corpus in priority order. The University deployment (Education sector) is the first production tenant; load Education first, then the universal XSC partition.

```
# Trigger ingestion job for Education sector (Sector 13, UCO-EDU nodes)
kubectl apply -f infra/kubernetes/jobs/rag-ingestion-education.yaml # Loads:
FERPA regulations, DOE guidance, institutional data governance templates #
Then load the universal XSC partition kubectl apply -f
infra/kubernetes/jobs/rag-ingestion-xsc.yaml # Loads: FLSA, OSHA, EEOC, IRS
Employment Tax, state data breach statutes (all 50 states)
```

Section 11 — Post-Deployment Validation Checklist

Complete all 24 items before declaring the deployment PRODUCTION READY. Items are listed in recommended execution order. Each item must be executed by the operator and result recorded in the deployment log.

#	Verification Step	Command / Script	Expected Result	Status
1	All Flyway migrations applied (V1-V4)	flyway info	All 4 rows = Success	✓
2	All 5 PostgreSQL roles created	SELECT rolname FROM pg_roles WHERE rolname IN (...)	5 rows returned	✓
3	UCO nodes seeded —	SELECT COUNT(*) FROM	350	✓

#	Verification Step	Command / Script	Expected Result	Status
	exactly 350 rows	uco_nodes		
4	UCO BLOCK/APPROVE/ESCALATE totals	Policy action count query by sector	BLOCK=192 / APPROVE=108 / ESCALATE=50	✓
5	No UCO nodes with risk_weight < 5	SELECT COUNT(*) WHERE risk_weight < 5	0	✓
6	Agency registry populated	SELECT COUNT(*) FROM agency_registry	>= 80 rows	✓
7	NAICS decoder populated	SELECT COUNT(*) FROM naics_decoder	>= 100 rows	✓
8	RAG Vault sector partitions present	SELECT COUNT(*) FROM rag_vault_sector_partitions	20 rows	✓
9	All 20 rag_chunks partitions exist	pg_inherits query (Section 10.2)	20 rows	✓
10	WORM triggers active on all 6 tables	scripts/db/verify_worm_triggers.sql	All 6 tables PASSED	✓
11	Ed25519 key — all 3 publication locations consistent	scripts/ops/verify_key_publication_consistency.py	CONSISTENT ✓	✓
12	WAL streaming replication active	SELECT * FROM pg_stat_replication	1 row, state=streaming	✓
13	Replication lag within RPO	replay_lag < INTERVAL '15 min'	True	✓
14	Gate 530 IPC	ls	Socket file	✓

#	Verification Step	Command / Script	Expected Result	Status
	socket active	/tmp/gate530.sock on each middleware-engine pod	exists	
15	Gate 530 fail-closed test	Submit request with UCO_DB_UNAVAILABLE flag	BLOCK returned	✓
16	Evidence Fabric signing test	POST /v1/infer with test payload	Evidence Package returned with valid Ed25519 signature	✓
17	Signature verification with DNS key	scripts/ops/verify_evidence_package.py --source dns	VERIFIED ✓	✓
18	Signature verification with filesystem key	scripts/ops/verify_evidence_package.py --source filesystem	VERIFIED ✓	✓
19	UCO vector embeddings populated	SELECT COUNT(*) FROM uco_nodes WHERE vector_embedding IS NOT NULL	350	✓
20	Merkle root publisher running	kubectl get cronjob merkle-root-publisher -n ios-plus	ACTIVE	✓
21	WORM integrity weekly job scheduled	kubectl get cronjob worm-integrity-check -n ios-plus	ACTIVE	✓
22	Health check endpoint live	GET /v1/admin/health	{"status": "healthy", "components": {...all green}}	✓
23	Prometheus metrics	GET /v1/admin/metrics	200 OK with metric values	✓

#	Verification Step	Command / Script	Expected Result	Status
	endpoint			
24	Multi-layer inference end-to-end test	scripts/ops/e2e_inference_test.py	Gate PASS, Evidence Package signed, latency < 380ms	✓

Production Readiness Gate — Hard Blockers

Do not declare the deployment PRODUCTION READY until all 24 checklist items show ✓. Items 3–5 (UCO seed verification) and 10 (WORM triggers) are **hard blockers** — a single failure on any of these five items constitutes a critical compliance failure and requires full rollback. Record the operator name and timestamp for each completed item in the deployment log.

Section 12 — Key Rotation Operational Procedure

12.1 90-Day Rotation Schedule

Key rotation is calendar-based (every 90 days), not usage-based. The first rotation occurs at Day 90 from the initial ceremony date. Subsequent rotations are managed by an automated Kubernetes CronJob that creates a Vault-generated rotation request; a human operator completes triple-publication within a 4-hour scheduled maintenance window.

Rotation Event	Trigger	Operator Action Required	Window
Initial ceremony	First deployment	Full air-gapped ceremony (Section 4)	Half-day maintenance window
Scheduled rotation	Every 90 days (calendar)	In-HSM generation + triple-publication	4-hour maintenance window
Emergency revocation	Compromise suspected	Immediate Vault revocation + incident response	No scheduled window — immediate

12.2 Rotation Steps

1. Generate new key pair in-HSM via Vault transit engine (no air gap required for rotation).
2. Publish new public key to all three locations (database, DNS TXT, filesystem) — atomic update per Section 4.4 procedure.
3. Set `activates_at = NOW() + 1 hour` on new key entry in `ios_signing_keys`.
4. Set `expires_at = NOW()` on previous key — no new signatures issued after this point.
5. Verify consistency across all three publication locations using `verify_key_publication_consistency.py`.
6. Log rotation event as a signed system event (the last act of the previous key is signing its own retirement record).
7. Run post-rotation WORM verification and Merkle root verification.

12.3 Emergency Revocation

If a signing key compromise is suspected at any time, execute the following steps in immediate sequence without waiting for a maintenance window.

8. Immediately revoke in Vault: `vault delete transit/keys/ios-evidence-signing` (version-specific revocation).
9. Set `revoked_at = NOW()` in the `ios_signing_keys` table for the affected key.

10. Update all three publication locations with revocation notice immediately.
11. Run evidence packages flag script: mark all Evidence Packages signed with the revoked key as requiring re-verification.
12. Notify security team and initiate incident response per the security runbook.

Revocation Impact

Emergency revocation invalidates all Evidence Packages signed by the affected key until re-verification is complete. Do not treat revocation as a routine operation — coordinate with Engineering Lead and Legal before acting on suspected compromise.

Section 13 — Observability and Alerting

13.1 Prometheus Alert Rules (Critical Alerts)

```
groups: - name: ios-plus-critical rules: - alert: Gate530Timeout
  expr: rate(gate530_evaluations_total{result="timeout"}[5m]) > 0 severity:
  critical annotations: summary: "Gate 530 evaluation timeout – fail-
  closed BLOCK returned" - alert: UCUNodesEmpty expr: uco_nodes_total <
  350 severity: critical annotations: summary: "UCO nodes table
  below 350 – Gate 530 compliance coverage degraded" - alert:
  WORMViolationAttempt expr: increase(worm_violation_attempts_total[1m]) >
  0 severity: critical annotations: summary: "WORM trigger fired
  – attempted modification of append-only audit table" - alert:
  ReplicationLagExceeded expr: pg_replication_lag_seconds > 900
  severity: critical annotations: summary: "WAL replication lag
  exceeds 15 minutes – RPO at risk" - alert: KeyPublicationInconsistency
  expr: ios_signing_key_publication_consistent == 0 severity: critical
  annotations: summary: "Ed25519 key publication locations inconsistent –
  investigate immediately"
```

Alert Name	Trigger Condition	Severity	Immediate Action
Gate530Timeout	Any Gate 530	Critical	Verify sidecar IPC

Alert Name	Trigger Condition	Severity	Immediate Action
	timeout in 5-minute window		socket; check Redis cluster health
UCONodesEmpty	UCO node count drops below 350	Critical	Halt traffic; re-seed from canonical CSV; verify WORM tables
WORMViolationAttempt	Any WORM trigger fired in 1-minute window	Critical	Identify source role/connection; review application logs
ReplicationLagExceeded	WAL replication lag > 900 seconds	Critical	Check network between primary and standby; check disk I/O on standby
KeyPublicationInconsistency	Key publication consistency check returns 0	Critical	Run manual consistency check; halt Evidence Fabric if mismatch confirmed

Section 14 — Document Lineage

This section maps each prior Engineering Body document to the operational steps in EB Doc 6 that execute its specifications. All six documents are required reading before executing this runbook.

Source Document	What EB Doc 6 Deploys From It	EB Doc 6 Section
EB Doc 1 — System Architecture & Module Design	Module structure and Kubernetes pod/sidecar architecture; node labels and resource specifications by tier	Sections 6 and 10
EB Doc 2 — Evidence	Ed25519 key ceremony	Sections 4 and 12

Source Document	What EB Doc 6 Deploys From It	EB Doc 6 Section
Fabric & Cryptographic Signing Architecture	procedure, triple-publication protocol, 90-day rotation schedule, and key revocation procedure	
EB Doc 3 — COS+ Database Schema & Access Control	Flyway migrations V1-V4 executed in sequence; PostgreSQL roles created and granted; UCO seed scripts loaded	Sections 2, 3, and 5
EB Doc 4 — Gate 530 UCO Compliance Dimension Matrix	350-node UCO dataset seeded and verified; BLOCK/APPROVE/ESCALATE policy action totals validated; Gate 530 fail-closed behavior tested	Sections 5 and 11
EB Doc 5 — RAG Vault Vector Search & Retrieval Design	20-partition HNSW index initialization; sector-specific ef_search parameters applied via Helm values; initial corpus loading for Education and XSC partitions	Sections 6 and 10
EB Doc 6 (this document) — Deployment, Operations & Recovery Runbook	The runbook that executes all of the above in operational sequence	All sections

Appendix A — Seed File Manifest

Seed File	Target Table	Row Count	SHA-256 Checksum	Last Updated
agency_registry.csv	agency_registry	>= 80	Verify at runtime: sha256sum db/seeds/agency_registry.csv	May 2026
naics_decoder	naics_decoder	>= 100	Verify at	May 2026

Seed File	Target Table	Row Count	SHA-256 Checksum	Last Updated
.csv			runtime: sha256sum db/seeds/naics_decoder.csv	
uco_nodes_350.csv	uco_nodes	350 (exact)	Verify at runtime: sha256sum db/seeds/uco_nodes_350.csv	May 2026
code_crosswalk.csv	code_crosswalk	>= 200	Verify at runtime: sha256sum db/seeds/code_crosswalk.csv	May 2026
compliance_chains.csv	compliance_chains	Varies by deployment	Verify at runtime: sha256sum db/seeds/compliance_chains.csv	May 2026

Checksum Verification Required

All seed files must be SHA-256 checksummed before loading and compared against the canonical checksums published in the SMEPro internal artifact registry. A checksum mismatch means the file has been modified since approval and must not be loaded. Escalate to Engineering Lead.

Appendix B — Rollback Procedures

Rollback is only permissible on fresh deployments with no live application traffic. Never roll back a production system with active Evidence Package data — contact Engineering Lead before any rollback on a system that has processed real transactions.

Migration	Rollback Script	Caveat
V4 → V3	db/rollback/ R4__undo_uco_schema_ext ension.sql	Drops all UCO tables and seed data. All UCO evaluation results lost. Medium risk.
V3 → V2	db/rollback/ R3__undo_rag_vault_sche ma.sql	Drops RAG Vault tables and HNSW indexes. All ingested corpus data lost. Low risk (no WORM data).
V2 → V1	db/rollback/ R2__undo_audit_tables.s ql	Destroys all WORM audit tables including evidence_packages, gate_decisions, merkle_roots. Only permissible on fresh deployments with zero production data. Requires dual-operator sign-off. This rollback is irreversible — all evidence is permanently destroyed.
V1 → Baseline	db/rollback/ R1__undo_initial_schema .sql	Drops all IOS+ tables. Returns database to empty state. Only permissible on fresh deployments.

V2 Rollback — Irreversible Data Destruction

Rolling back V2 permanently destroys all WORM audit tables and any data written to them. This action is only permissible during initial deployment before any evidence data has been written. On any system that has processed production traffic, V2 rollback is **forbidden**. Requires written dual-operator authorization and Engineering Lead approval before execution.

Appendix C — Environment Variables Reference

Variable	Consumed By	Description	Source
COS_DB_URL	All services, CLI scripts	PostgreSQL JDBC/psql connection string for COS+ primary	Vault: secret/ios-plus/db/connection
DB_HOST	Flyway, psql scripts	COS+ primary hostname or IP	Kubernetes ConfigMap / Vault
PRIMARY_HOST	pg_basebackup, recovery scripts	PostgreSQL primary hostname for replication	Kubernetes ConfigMap
PRIMARY_URL	Recovery scripts	psql-format connection string for primary	Vault
STANDBY_URL	Recovery scripts	psql-format connection string for standby	Vault
NEW_PRIMARY_URL	Post-failover scripts	Connection string for newly promoted primary after failover	Vault (updated post-failover)
NEW_PRIMARY_HOST	Recovery scripts, Vault patch	Hostname of newly promoted primary	Set by operator post-failover
FLYWAY_USER	Flyway CLI	Database user for migration execution (cos_admin)	Shell environment
ZONE_ID	Key ceremony DNS publication	Route53 hosted zone ID for DNS TXT record publication	Vault: secret/ios-plus/dns/zone_id
KEY_ID	Key ceremony scripts	UUID v4 for the current active signing key	Generated by ceremony script; stored in Vault
PUBLIC_KEY_B64URL	Key ceremony scripts	Base64url-encoded Ed25519 public key bytes	Generated by ceremony script; stored in Vault
FINGERPRINT	Key ceremony scripts	SHA-256 fingerprint of public key bytes	Generated by ceremony script; stored in Vault
EXPIRES_UNIX	DNS TXT record publication	Unix timestamp of key expiry (NOW	Computed at ceremony time

Variable	Consumed By	Description	Source
		+ 91 days)	
ACTIVATES_AT	Key metadata record	ISO-8601 timestamp when key becomes active (NOW + 1 hour)	Computed at ceremony time
EXPIRES_AT	Key metadata record	ISO-8601 timestamp when key expires (NOW + 91 days)	Computed at ceremony time
OPERATOR_NAME	Key ceremony SQL	Name of operator completing the ceremony (for audit log)	Set by operator before ceremony

Appendix D — Glossary

Term	Definition
4-eyes principle	A security control requiring that any sensitive action (such as the Ed25519 key ceremony) is witnessed and co-authorized by a second operator. No single individual can complete the action alone.
Air-gapped ceremony	A key generation procedure performed on a machine with all network interfaces physically disabled or removed, ensuring the private key seed is never exposed to a network at any point during generation.
ef_construction	HNSW index build parameter controlling the size of the candidate list during index construction. Higher values produce better recall at the cost of longer build time. Set per-sector in EB Doc 5.
ef_search	HNSW query-time parameter controlling the size of the dynamic candidate list during approximate nearest neighbor search. Higher values improve recall at

Term	Definition
	the cost of query latency. Set per risk tier in <code>values.yaml</code> .
Fail-closed	A failure mode design where a timeout, error, or unavailability in a dependency causes the system to return the most restrictive safe response (BLOCK) rather than permitting the action. Gate 530 is fail-closed by design.
Flyway	Database schema migration tool used to apply and version-control SQL migrations (V1-V4) against the COS+ PostgreSQL database. Ensures migrations are applied in order and tracked in the <code>flyway_schema_history</code> table.
HNSW	Hierarchical Navigable Small World — a graph-based approximate nearest neighbor index algorithm used by pgvector for vector similarity search in the RAG Vault. Provides sub-linear query time with tunable recall.
Patroni	High-availability solution for PostgreSQL that manages automatic primary election, failover, and replication topology. Used in the IOS+ COS+ cluster for automated primary promotion during unplanned failures.
pgvector	PostgreSQL extension providing vector data types, HNSW and IVFFlat indexes, and vector similarity operators (<code><-></code> , <code><=></code> , <code><#></code>). Used by RAG Vault for UCO node and regulatory corpus similarity search.
PITR (Point-in-Time Recovery)	PostgreSQL recovery mechanism that replays WAL segments from a base backup to restore the database to any specific past timestamp, enabling recovery from accidental DDL or data corruption.
Replication slot	A PostgreSQL feature that tracks which WAL data a standby has consumed, ensuring the primary retains WAL segments until the standby confirms receipt. Prevents WAL from being removed before the standby has

Term	Definition
	applied it.
RPO (Recovery Point Objective)	The maximum tolerable period of data loss measured from the last confirmed good state to the point of failure. The IOS+ COS+ cluster RPO is 15 minutes, enforced by <code>archive_timeout = 900</code> in <code>postgresql.conf</code> .
RTO (Recovery Time Objective)	The maximum tolerable time to restore service after an outage. The IOS+ COS+ cluster RTO is 90 seconds for unplanned primary failure, achieved via Patroni automatic failover.
Triple-publication	The IOS+ protocol requiring that every Ed25519 signing key's public key is written to three independent, authoritative locations simultaneously: the COS+ database, a DNS TXT record, and the deployment filesystem. Consistency across all three is continuously monitored.
UCO node seed	The canonical 350-row dataset representing all compliance dimensions across 19 industry sectors plus the XSC cross-cutting partition, loaded from <code>uco_nodes_350.csv</code> into the <code>uco_nodes</code> table. The seed is the authoritative source of truth for Gate 530 policy evaluation.
Vault transit engine	A HashiCorp Vault secrets engine that provides encryption-as-a-service. Used in IOS+ to manage Ed25519 signing keys in hardware-protected storage where the key material never leaves Vault unencrypted.
WAL (Write-Ahead Log)	PostgreSQL's transaction durability mechanism. All changes are written to WAL before being applied to data files. WAL is also the transport mechanism for streaming replication and point-in-time recovery.
WORM (Write Once Read Many)	An immutability model enforced at the database trigger level. All audit tables in IOS+ are WORM-protected: INSERT is permitted, UPDATE and DELETE raise <code>restrict_violation</code> . This ensures

Term	Definition
	evidence records cannot be tampered with after creation.